

Security Audit Report - circomlib circuits/

Field	Value
Target	iden3/circomlib, the circuits/ library
Version	v2.0.5 . commit 35e54ea
Scope	All 57 .circom files under circuits/ (R1CS-level soundness / completeness) BN254 scalar field $p = 21888242871839275222246405745257275088548364400416034343698204186575808495617$ ($2^{253} < p < 2^{254}$); EC operations on Baby Jubjub (cofactor = 8)
Curve / Field	File-by-file, constraint-by-constraint review; dedicated audit of every <-- (unconstrained assignment); downstream misuse patterns sampled across real projects via Sourcegraph; all findings reproduced with runnable witness-level PoCs (Appendix B)
Method	
Date	2026-06-11
Auditor	Claude Opus 4.8 (AI-conducted audit)

0. Executive Summary

circomlib is the most widely reused low-level circuit library in the circom ecosystem. This file-by-file audit answers two questions directly:

Question 1 - What is wrong with circomlib's own implementation?

circomlib's internal self-use is self-consistent and safe - we found no backdoor by which the library, used the way it uses itself, yields a directly forgeable proof. However, the library makes a systematic design trade-off: many templates do not enforce their own security preconditions (point is in the prime-order subgroup, bit decomposition is canonical/unique, inputs are in range, selectors are binary). Instead these preconditions are implicitly delegated to the caller. We call this "constraint-boundary externalization." It is not a bug per se, but it makes the library extremely easy to misuse when integrated. Consolidated into 6 findings (F-1 ... F-6).

Question 2 - What must integrators watch out for?

The caller must actively re-establish the preconditions the library does not enforce; otherwise the library's "safe assumption" becomes the caller's exploitable vulnerability. The three highest-frequency, highest-impact misuses are: (a) using Num2Bits(254/256) for uniqueness/range; (b) feeding an attacker-controlled, unvalidated point into EscalarMulAny; (c) using the legacy bit-IO eddsa.circom, or feeding raw prover bits into bit-consuming primitives. Part II gives a checkable caller checklist with real-world examples (who did it right, who matched the dangerous pattern).

Findings Overview

ID	Module	Type	Severity	Directly exploitable inside the library?	Consequence if a caller misuses it
F-1	MontgomeryAdd / MontgomeryDouble	Under-constrained at exceptional points	High	No	Variable-base scalar mul output can be forged
F-2	Num2Bits (non-strict)	Aliasing / non-injective	High	No	Uniqueness/range bypass (double-spend, mint)
F-3	LessThan & comparators	No range constraint on inputs	Medium	No	Range/interval check bypass

ID	Module	Type	Severity	Directly exploitable inside the library?	Consequence if a caller misuses it
F-4	Mux / Switcher / Multiplexor2	Selector not constrained binary	Medium	No	Output becomes arbitrary linear interpolation
F-5	Bits2Point / Point2Bits	Empty templates, unconstrained output	Low	Yes	Soundness collapses the moment it is called
F-6	Bit-consuming primitives + legacy eddsa.circom	Bit inputs not forced in {0,1}	High (legacy EdDSA)	Yes (legacy wires prover bits directly)	Signature forgery -> spend others' assets

Part I - Problems in circomlib's Own Implementation

This part answers "what is wrong with the library itself." Conclusion: no internally-self-exploitable backdoor, but 6 instances of precondition externalization. Each is given with root cause, exploitation condition, and current library status.

F-1 | High (conditional soundness) - MontgomeryAdd / MontgomeryDouble under-constrained at exceptional points

File: circuits/montgomery.circom

```
lamda <-- (in2[1] - in1[1]) / (in2[0] - in1[0]);
lamda * (in2[0] - in1[0]) == (in2[1] - in1[1]);
out[0] <== B*lamda*lamda - A - in1[0] - in2[0];
out[1] <== lamda * (in1[0] - out[0]) - in1[1];
```

The only constraint on lamda is $\text{lamda}(\text{in2}[0] - \text{in1}[0]) == (\text{in2}[1] - \text{in1}[1])$. When the two input points share an x-coordinate ($\text{in2}[0] == \text{in1}[0]$):

- Equal points ($\text{in2}[1] == \text{in1}[1]$): the constraint degenerates to $\text{lamda} == 0$, satisfied for any lamda -> out may take arbitrary values. Genuinely under-constrained; multiple valid witnesses exist.
- Adding the negation ($\text{in2}[1] != \text{in1}[1]$): the constraint becomes $0 == \text{nonzero}$, unsatisfiable -> completeness failure (DoS).

MontgomeryDouble is analogous: when $\text{in}[1] == 0$ (a 2-torsion point), lamda is entirely free. This is the classic "incompleteness" of the Montgomery addition formula.

Library status:

- Fixed-base paths are safe. Pedersen and EscalarMulFix use constant generators; the author argues in mux4.circom comments that the accumulator and window outputs are never equal nor zero (accumulator $\geq 2^{247} \cdot B$, window $\leq 2^{246} \cdot B$). This review concurs with that structural argument.
- Variable-base path is at risk. escalarmulany.circom states // This function assumes that p is in the subgroup and it is different to 0, but this precondition is not enforced in-circuit (only $p == 0$ is handled). If a caller feeds an attacker-controlled point without a subgroup check, the internal 2P and accumulator may collide into equal points for certain scalar-bit combinations, triggering the under-constraint.

Reproduced in Appendix B.1.

F-2 | High (footgun -> downstream soundness) - Num2Bits (non-strict) aliasing

File: circuits/bitify.circom

```
template Num2Bits(n) {
```

```

for (var i = 0; i<n; i++) {
  out[i] <-- (in >> i) & 1;
  out[i] * (out[i] - 1) === 0;
  lc1 += out[i] * e2;  e2 = e2+e2;
}
lc1 === in;
}

```

The constraints only guarantee "weighted bit sum == in (mod p)." When $2^n > p$ (typically $n=254/256$), Num2Bits is non-injective: $2^{254} - p \approx 7.05e75$, so every small in with $in + p < 2^{254}$ (about 32% of the field) has two valid 254-bit decompositions (the bits of in and the bits of in+p).

Consequence: any downstream circuit using Num2Bits(254/256) for a "canonical unique bit commitment" (nullifier/commitment) or a "range check" can be bypassed by a malicious prover supplying the second representation.

Library status: the library provides Num2Bits_strict() (which adds AliasCheck) as the correct primitive, but the default Num2Bits has no guardrail and no warning in its name. Note: when n is well below 253 (e.g. Num2Bits(120) for range checks) no aliasing exists - that is a safe use.

Reproduced in Appendix B.2.

F-3 | Medium (footgun) - comparators do not range-check their inputs

File: circuits/comparators.circom

```

template LessThan(n) {
  assert(n <= 252);
  component n2b = Num2Bits(n+1);
  n2b.in <== in[0] + (1<<n) - in[1];
  out <== 1 - n2b.out[n];
}

```

LessThan(n) (and LessEqThan / GreaterThan / GreaterEqThan) is only correct when both inputs are $< 2^n$. `assert(n<=252)` is a compile-time bit-width limit and imposes no runtime range constraint on `in[0]` / `in[1]`. Passing $\geq 2^n$ (especially values near p) makes `in[0]+(1<<n)-in[1]` wrap around in the field; `out` returns a wrong result while the circuit remains satisfiable -> the interval test can be bypassed. `IsZero` / `IsEqual` / `ForceEqualIfEnabled` are themselves sound.

Reproduced in Appendix B.3.

F-4 | Medium (footgun) - Mux / Switcher / Multiplexor2 do not constrain selectors to be binary

Files: switcher.circom, escalarmulany.circom, mux1~4.circom

Switcher, Multiplexor2, and MultiMux1/3/4 all assume the selector in $\{0,1\}$, but none of them contains `sel(sel-1)===0`:

- MultiMux1: `out[i] <== (c[i][1]-c[i][0])s + c[i][0]`, s unconstrained.
- MultiMux3 / MultiMux4: expanded via products like `s10=s[1]s[0]`; equivalent to a table lookup only when s is binary, otherwise arbitrary.
- Switcher: `aux<==(R-L)sel`; with non-binary sel the outputs are an arbitrary linear interpolation of L and R.

Library status: the library's internal uses (SMT, EdDSA, Pedersen, EscalarMul) all derive selectors from Num2Bits outputs (already binary), hence safe.

Reproduced in Appendix B.4.

F-5 | Low / Info - Bits2Point and Point2Bits are empty templates

File: circuits/pointbits.circom

```
template Bits2Point() { signal input in[256]; signal output out[2]; } // no body
template Point2Bits() { signal input in[2]; signal output out[256]; } // no body
```

The two non-`_Strict` versions are completely unconstrained; out is a free signal. This is the only implementation in the library that collapses the moment it is called (the other 5 require a malicious caller-supplied value to trigger). The real implementations are `Bits2Point_Strict` / `Point2Bits_Strict`.

Side note: in `Bits2Point_Strict`, when y corresponds to $x=0$, $\text{sign}(0)=0$; if $\text{in}[255]==1$ then $\text{signCalc.out}===\text{in}[255]$ is unsatisfiable - a minor completeness DoS, not a soundness issue.

F-6 | High (legacy EdDSA) - bit-consuming primitives do not force inputs in {0,1}; legacy eddsa.circom wires prover bits directly

Many "bit-input" primitives assume their inputs are already in $\{0,1\}$ but do not enforce it internally: `Bits2Num`, `CompConstant`, `AliasCheck`, `Sign`, `Pedersen` (`Window4`), the `Sha256` family (`Xor3/Maj/Ch`), `EscalarMulFix.e[]`, `EscalarMulAny.e[]`, and the `MultiMux` family. Almost all internal callers feed `Num2Bits` outputs (already binary), hence safe.

The single dangerous instance is the legacy Pedersen-based verifier `circuits/eddsa.circom::EdDSAVerifier`: it wires the raw signal input `S[256]` / `A[256]` / `R8[256]` directly into these primitives with no $x(x-1)===0$ anywhere:

```
component compConstant = CompConstant(<subgroup order - 1>);
for (i=0; i<254; i++) S[i] ==> compConstant.in[i];
compConstant.out === 0; S[254]===0; S[255]===0; // only top 2 bits constrained
...
component mulFix = EscalarMulFix(256, BASE8);
for (i=0; i<256; i++) mulFix.e[i] <== S[i]; // S used directly as scalar bits
```

If `S[i]` is non-binary, then (a) the `CompConstant` "`S < subgroup order`" check can be bypassed, and (b) `S.B8` is computed incorrectly -> when `S` is prover-controlled, signature soundness is broken. `A` and `R8` are likewise not bit-constrained (`Bits2Point_Strict` only does a `BabyCheck` on the reassembled point and constrains the sign bit; it does not constrain the binariness of `in[0..253]`), and the same bits are also fed into the Pedersen hash, creating an exploitable representation-inconsistency surface.

Contrast: the modern versions fixed it. `eddsamimc.circom` and `eddsaposeidon.circom` instead take a field element `S`, re-decompose it internally via `Num2Bits(253)` (which forces binariness) before feeding `CompConstant` / `EscalarMulFix`, and use `Num2Bits_strict()` for `h`. This defensive rewrite is itself evidence of the legacy version's fragility.

Reproduced in Appendix B.5 (root cause: EscalarMulFix accepts non-binary scalar bits, and its output is determined by the bit pattern rather than the scalar value).

Confirmed Sound After Constraint-by-Constraint Review

i.e. parts where the library's own implementation has no problem (under the in-library-satisfied premise that input bits are already binary).

- Arithmetic hashes (all `<==`, no `<--`, fully constrained): `Poseidon` / `PoseidonEx`, `MiMC7` / `MultiMiMC7`, `MiMCSponge` / `MiMCFeistel`, `poseidon_old`, `SMTHash1/2`, `smthash_mimc`.
- `IsZero` / `IsEqual` / `ForceEqualIfEnabled`: classic constructions, fully constrained.
- `BinSum` / `BinSub`: `lin===lout` plus per-bit binary constraints, correct.
- `CompConstant` / `AliasCheck` / `Sign`: logically correct; `AliasCheck` uses `CompConstant(-1)` to correctly exclude

>= p aliases (premise: input bits already binary, see F-6).

- BabyAdd / BabyCheck / BabyDbl: twisted-Edwards complete addition formula; denominators $1 \pm d \cdot \tau$ are never zero inside the subgroup - no F-1 exceptional-point issue.

- Multiplexer / Decoder / EscalarProduct: $\text{out}[i](\text{inp}-i) \equiv 0 + \text{success binary} + \text{dec.success} \equiv 1$, correct.

- gates.circom (XOR/AND/OR/NOT/NAND/NOR/MultiAND): correct arithmetic implementations (inputs must be binary).

- SHA-256 chain: outputs are constrained binary via BinSum; $\text{out}[i] \leftarrow \text{outCalc}[i]$ is re-pinned by subsequent $\equiv$ (inputs must be binary, see F-6).

- SMT (SMTVerifier / SMTProcessor / SMTLevIns): state-machine arithmetic expansion, terminal $\equiv 1$, key bits via Num2Bits_strict, Switcher selectors from already-binary bits - rigorously designed.

- EdDSAPoseidonVerifier / EdDSAMiMCVerifier: $S < \text{subgroup order}$, x_8 into subgroup, $A \neq 0$, terminal ForceEqualIfEnabled - structurally complete.

- EscalarMul / EscalarMulWindow / EscalarMulW4Table: fixed-base windowed method, BabyAdd complete addition.

Appendix A - Reviewed Files (57)

```
aliascheck babyjub binsub binsum bitify comparators compconstant
eddsa eddsamimc eddsamimcsponge eddsaposeidon
escalarmul escalarmulany escalarmulfix escalarmulw4table
gates mimc mimcsponge montgomery multiplexer
mux1 mux2 mux3 mux4 pedersen pedersen_old
pointbits poseidon poseidon_constants poseidon_constants_old poseidon_old
sign switcher
sha256/{ch,constants,main,maj,rotate,sha256,sha256_2,sha256compression,
      sha256compression_function,shift,sigma,sigmaplus,t1,t2,xor3}
smt/{smthash_mimc,smthash_poseidon,smtlevins,smtprocessor,smtprocessorlevel,
      smtprocessorm,smtverifier,smtverifierlevel,smtverifiersm}
```

Part II - What Integrators Must Watch Out For

Core principle in one sentence: circomlib externalizes its security preconditions to you; you must put them back.

Caller Safety Checklist (mapped to each finding)

#	What you are doing	What you must do	Finding
1	Using Num2Bits(n)	If $n \geq 253$ and used for uniqueness/range/commitment -> use Num2Bits_strict; for pure range checks use $n < 253$ (e.g. 120)	F-2
2	Using EscalarMulAny (variable base)	At the entry, apply BabyCheck (on-curve) + cofactor/subgroup check, especially when the point is external/attacker-controlled	F-1
3	Using LessThan/GreaterThan/LessEqThan/GreaterEqThan	Add Num2Bits(n) range constraints on both operands so both are $< 2^n$	F-3
4	Using Switcher/MultiMux/Multiplexor2	Ensure the selector comes from a Num2Bits output, or add $\text{sel}(\text{sel}-1) \equiv 0$ explicitly	F-4
5	Converting pointbits	Never call the empty stubs Bits2Point/Point2Bits; only use the _Strict versions	F-5

#	What you are doing	What you must do	Finding
6	Verifying EdDSA	Abandon the bit-IO eddsa.circom; use the field-IO	F-6
7	Using Pedersen/Sha256 etc. directly	eddsaposeidon/eddsamimc Before feeding, constrain each bit with $b(b-1)===0$ or produce it via Num2Bits	F-6
8	Any public signal (nullifiers, etc.)	Enforce input < SNARK_SCALAR_FIELD on the contract side to prevent n vs n+p malleability double-spend	cross-layer
9	Value-conservation circuits	Range checks must close the inductive invariant (all tree leaves bounded); cross-layer (uintXXX in contract) must match the circuit bit-width	cross-layer

Misuse -> Consequence Matrix

Misuse pattern	Direct consequence	Exploitable as
Num2Bits(254/256) for nullifier uniqueness	Same note yields two distinct nullifier bit-strings	Double-spend
Num2Bits(254/256) for range/balance	Value larger than field order passes the range check	Mint / theft
Raw EscalarMulAny on attacker point	Scalar-mul output under-constrained, freely chosen	Forged shared secret / signature component
bit-IO eddsa.circom wired to prover bits	Non-binary S/A/R8 bypass subgroup check and scalar-mul	Signature forgery -> spend others' assets
Non-binary selector into Switcher	Merkle path / branch becomes linear interpolation	Forged Merkle membership proof

Appendix B - Reproducible Proofs of Concept

All findings were reproduced as witness-level PoCs using the actual circomlib sources. Methodology: circom's witness generator aborts on any unsatisfied $===$ constraint, so if a malicious input successfully generates a witness, the R1CS has accepted an assignment it should have rejected - a decisive demonstration of the under-constraint / bypass. Each PoC pairs an attack input with a control input (an honest case) to rule out a mis-modeled circuit.

Artifacts live under poc/ (.circom sources, input JSONs, compiled _js/ witness generators). Reproduce any case with:

```
cd poc
circom <name>.circom --wasm -o .
node <name>_js/generate_witness.js <name>_js/<name>.wasm <input>.json out.wtns
# exit 0 = R1CS accepted the input; non-zero (Assert Failed) = rejected
```

B.1 - F-1: MontgomeryAdd under-constrained at equal points

poc/f1_montadd.circom replicates MontgomeryAdd's constraints but exposes lamda as a prover input (modeling a malicious prover who does not follow the honest witness generator). With $A=168698$, $B=1$ and $in1==in2==(5,7)$:

Input	lamda	Result
in_f1_w1.json	0	accepted - out = (168708, 7)
in_f1_w2.json	3	accepted - out = (168699, ...)

Input	lamda	Result
in_f1_honest_bad.json (distinct points (5,7),(9,11), wrong lamda=0)	0	rejected

Two different out values satisfy the constraints for the same $in_1 == in_2 \rightarrow$ the output is under-constrained. The control (distinct points) pins lamda uniquely and rejects a wrong value, proving the freedom is specific to exceptional points.

B.2 - F-2: Num2Bits(254) aliasing / non-injective

poc/f2_vuln.circom exactly replicates the constraints Num2Bits(254) imposes (each bit binary + weighted sum $==$ num), with the 254 bits as input. poc/f2_strict.circom additionally runs AliasCheck (i.e. Num2Bits_strict).

Circuit	bits(5)	bits(5+p)
f2_vuln (replicates Num2Bits)	accepted [PASS]	accepted [PASS]
f2_strict (adds AliasCheck)	accepted [PASS]	rejected [FAIL]

Two distinct bit-arrays both represent $num=5$ under the non-strict relation $\rightarrow$ non-injective. If those bits are used as a nullifier, one note yields two valid nullifiers $\rightarrow$ double-spend. AliasCheck (strict) rejects the aliased representation.

$5+p = 21888242871839275222246405745257275088548364400416034343698204186575808495622$, which is $< 2^{254}$ and $= 5 \pmod{p}$.

B.3 - F-3: comparator range bypass

poc/f3_less-than.circom is a "prove I am below the limit" circuit asserting $value < limit$ via LessThan(8).

Input	Honest truth	Result
value=3, limit=5	$3 < 5$ true	accepted [PASS]
value=10, limit=5	$10 < 5$ false	rejected [FAIL] (gadget works for in-range)
value=p1, limit=5	$p1 < 5$, false	accepted [PASS] (BUG)

The circuit accepts the claim " $p1 < 5$." The control (value=10) failing proves the gadget is correct for in-range inputs; the attack passing proves the out-of-range bypass.

B.4 - F-4: Switcher non-binary selector

poc/f4_vuln.circom uses circomlib's Switcher with $L=0$, $R=1$ (so $outL = sel$) and asserts $outL == 2$. poc/f4_fixed.circom adds the caller's $sel(sel-1) == 0$.

Circuit	sel=2
f4_vuln (bare Switcher)	accepted [PASS] - $outL=2$, outside the legal set $\{0,1\}$
f4_fixed (binary constraint added)	rejected [FAIL]

A non-binary selector drives the output to an out-of-set value; adding the binary constraint (what the caller must do) rejects it.

B.5 - F-6: bit-IO EdDSA root cause - EscalarMulFix accepts non-binary scalar bits

eddsa.circom computes $S.B8$ via EscalarMulFix(256, BASE8) with no binary constraint on S . Two PoCs:

(a) Non-binary bits are accepted (poc/f6_accept.circom, single EscalarMulFix, no extra assertion):

Input	Result
$e = \{e[0]=2\}$ (non-binary)	accepted [PASS]
$e = \{e[0]=7, e[3]=5\}$ (arbitrary integers)	accepted [PASS]

EscalarMulFix contains no $b(b-1) \equiv 0$ - arbitrary non-binary scalar bits pass.

(b) Output is determined by the bit pattern, not the scalar value (poc/f6_escmulfix.circom, two EscalarMulFix instances asserted equal):

Input	Result
e1=bits(2), e2=bits(2) (control)	accepted [PASS] (equal outputs)
e1=bits(2), e2={e[0]=2} (same scalar value 2, different distribution)	rejected [FAIL] (outputs differ)

The canonical bits(2) and the non-binary {e[0]=2} have the same weighted value (2) but produce different S.B8. Therefore a prover supplying non-binary S can steer S.B8 to a non-canonical point, decoupled from the range-checked scalar -> it can satisfy the verification equation $S.B8 \equiv R8 + h.A8$ for forged (S, R8, A). This is the root cause of forgeability in the legacy bit-IO eddsa.circom.

Verification levels

Level	Description	Status
L0 reasoning	Argue from reading constraints	[PASS] Part I
L1 witness-level	Malicious witness satisfies R1CS + control group	[PASS] F-1 / F-2 / F-3 / F-4 / F-6 (this appendix)
L2 proof-level	Full Groth16 setup->prove->verify, producing an actually-verifying forged proof	Pending (requires snarkjs)
L3 against a real deployment	Reproduce against the target project's compiled circuit + on-chain verifier	Pending (requires their artifacts)